Terrain Generation and Optimisation

An Exploration of Complexity

Marshall Sharp

Falmouth University Games Academy

Abstract

Introduction

Main Objectives

1. Explore and research how to generate a mesh using graphics.
2. Create a tool for visualising terrain through a mesh.
3. Optimise the tool for lower end machines.

Materials and Methods

0.1 Terrain generation: The basics

Terrain generation works utilising the same system as Mesh generation, which takes an array of vertex's and indicies, passing them through the VAO, VBO and EBO buffers in conjunction with the Depth buffer.
The method used will be similar to this, though won't rely on directly communicating to OpenGL, which is the more optimal method. For this purpose, Unity has the GL library, which allows unity to communicate with OpenGL and render things using the GPU. For this, The program will communicate with the

Figure 1: An OpenGL render, utilising code from [?]

Optimization tools

For optimization, what is used is NVIDIA nsight, which is an application for debugging C++ projects. One issue this has is you cannot debug the unity project, due to it using C#. However, what can be done is debugging the build. In order to accomplish this, the developer must create a empty C++ project, and build the game into that project. They then must select Nsight to run the games .exe file. This will cause the unity build to open with the Nsight debugger enabled.

To optimise the project, it will be done in engine utilising the Profiler tool. The tool tracks Rendering, CPU usage, Memory/RAM usage, along with FPS. it function is similar to the diagnostic tools built into Visual Studio, where it pauses/breaks the program to get CPU and memory performance.

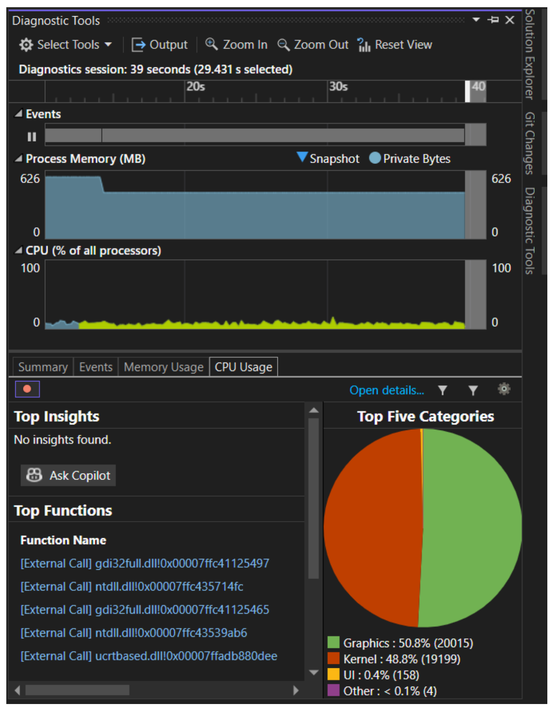

Figure 2: The Diagnostic tools within Visual studio.

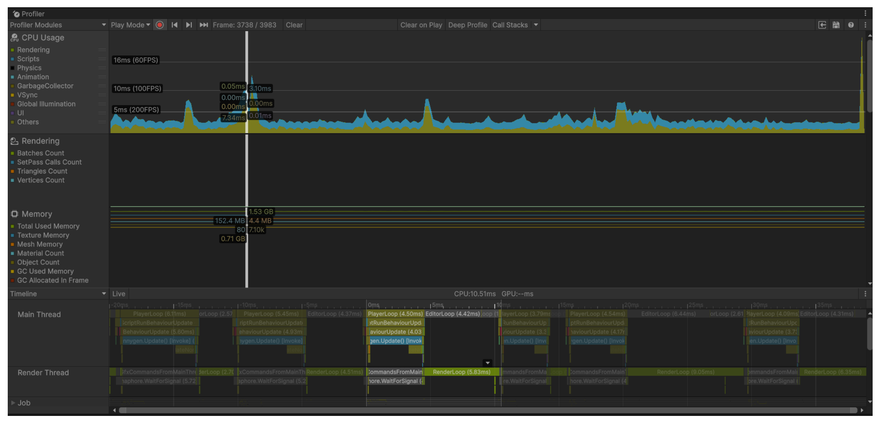

Figure 3: the profiler within Unity

Results

The tool, by default, runs at 145FPS using a 4050 laptop GPU and as seen in figure 4, and runs at approximately 55fps with every value set to it's maximum, in figure 5. The build used for the GPU optimisation goes up to 450. The tool also renders in very low poly's, and so for a cleaner terrain you would need to subdivide the generated mesh. Without this, it can lead to stretched faces and, if it has collision, impassible or hard to navigate terrain. The solution to this situation was to increment the verticies whenever it obtained the data to generate the mesh.

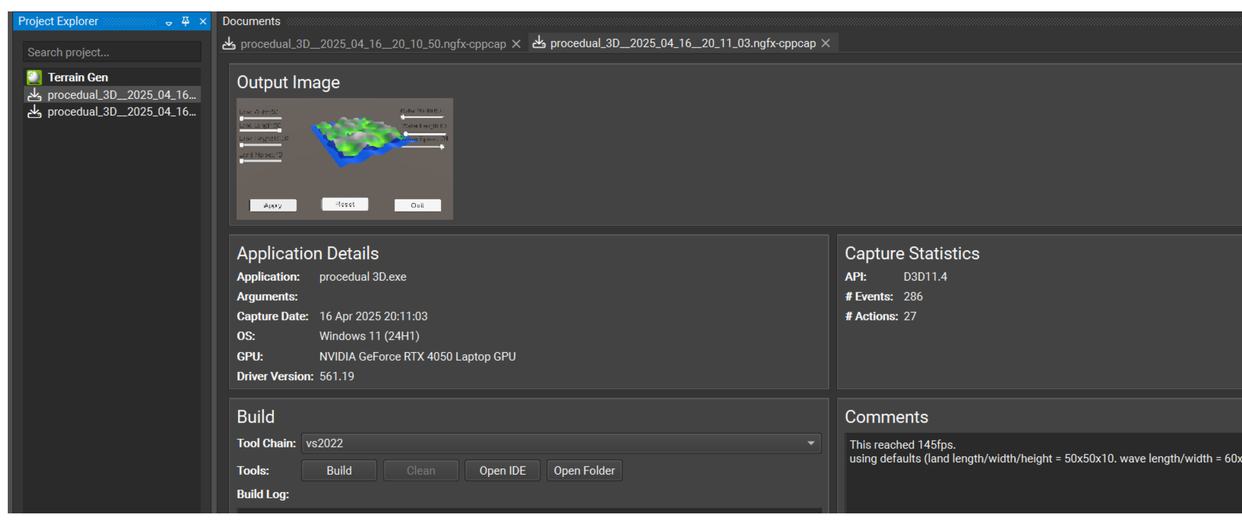

Figure 4: NSight result with the FPS noted. Using default values.

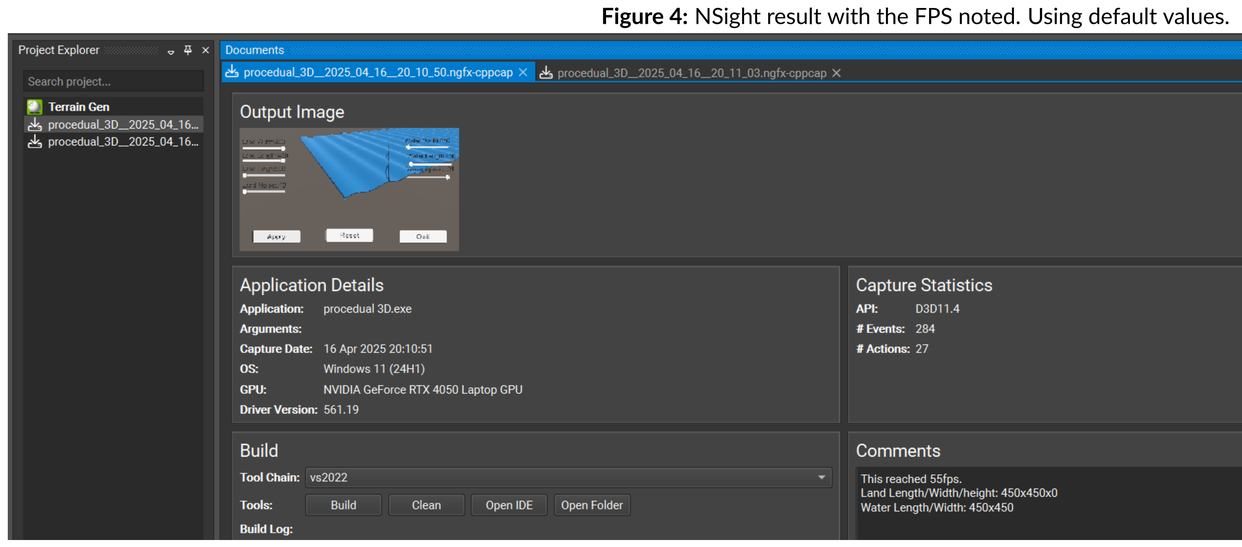

Figure 5: NSight result with the FPS noted. Values are at a maximum

Conclusions

On this poster, background on mesh generation is given and how it can be applied to terrain, in addition, methods of optimsations on the GPU are researched and used. Further research in this field will be done with culling algorithms in a Generated City using the GL libary for Unity.

References

[1] Se gon Roh and Hyouk Ryeol Choi. Differential-drive in-pipe robot for moving inside urban gas pipelines. *IEEE Transactions on Robotics*, 21(1):1–17, 2005.

[2] A. B. Jones and J. M. Smith. Article Title. *Journal title*, 13(52):123–456, March 2013.

[3] J. M. Smith and A. B. Jones. *Book Title*. Publisher, 7th edition, 2012.

[4] Danny Staple. *Learn Robotics Programming: Build and control AI-enabled autonomous robots using the Raspberry Pi and Python*. Packt Publishing Ltd, feb 12 2021. [Online; accessed 2025-04-10].

Acknowledgements